

Yet another **Vim** Introduction

YaVI v1.0

HARALD LICHTENSTEIN

May 3, 2026

Contents

1	Intention	5
2	In motion	9
2.1	Jumping around	9
2.2	Scrolling around	12
3	Working with text	15
3.1	Deleting text	16
3.2	Formatting text	18
3.3	Moving text	19
4	Hide and seek	23
4.1	Vim is case sensitive	31
4.2	Replacing	32
5	Working with files	35
5.1	Opening files from the terminal	35
5.2	Open files inside Vim	37
5.3	Start writing	38
5.4	Closing files	38
6	Working with registers	41
6.1	Some types of registers	41
6.2	Using macros	42
7	Working with windows	45
7.1	Split the screen	45
7.2	Diff mode	47
7.3	Commands for all windows	48

8	Features I love	51
8.1	Abbreviations	51
8.2	Counting and calculating	51
8.3	Features for programmers	52
8.4	Key combinations to think about	54
8.5	Magic g	55
8.6	Pasting search results	56
8.7	Reading external stuff	56
8.8	Repeating commands	57
8.9	Special deletions	57
8.10	Special yanking	58
8.11	Tricky tabulators	59
8.12	Using the terminal	60
8.13	Visual block mode	61
8.14	Word completion	63
9	My configuration	65
10	Bibliography	69

List of summaries

Meaning of the key symbols	7
Motion summary	12
Scroll summary	14
Insert summary	16
Delete summary normal mode	17
Delete summary command mode	17
Delete summary insert mode	17
Indentation options summary	18
Line wrap summary	19
Move summary	21
Move and copy summary	21
Special characters	26

More special characters	30
Even more special characters	31
Ranges summary	33
Buffers summary	38
Window commands summary	47
diff mode summary	48
Fold summary	54
g summary	56
Visual modes summary	63
Completion summary	64

1 Intention

Even though I have been using Vim for a long time now, I can't remember all these extremely efficient commands. This paper may serve as introduction for those who have already worked with Vim and as quick reference.

What do I do and where do I use this text editor? For work I do programming in several languages like Python, SQL, Markdown. At home I use Linux and L^AT_EX, so there is a lot of text to manipulate and since I use Vim I do it faster and with more fun. Most of the time there exists already some text file and I already know what has to be changed. The main focus is to find this part of text, which has to be changed and to modify it efficiently. Regular expression take a big part of this YaVI, because they help to do the job precisely and fast.

I have nothing to say about versions with graphical user interface and I will not use the mouse. I assume all commands are entered directly into a terminal or console, pure and simple.

This is an introduction and following topics are not part of it:

Tag stacking, Tabbed editing, Compiling and Quickfix, Plugins,
Scripting, IDE, Shells

So, if you want to use this introduction, you should open a terminal and enter Vim directly or Vim with a file name.

The power of Vim is nicely hidden in the various modes of operation. There are following modes:

- *normal* mode

This is the mode when starting Vim or coming back from other modes by pressing **Esc**. In this mode you do manipulations with Vim like moving the cursor or you delete, copy portions of text. The commands are executed without hitting **↵**.

- *command* mode

You have to type **:** or **/** to get into this mode. You also delete, copy, move and replace portions of text. You open or close files, you search and replace. These commands are executed by pressing **↵**.

- *insert* mode
In this mode you simply add text.
- *replace* mode
This mode is to overwrite what was already there
- *visual* mode and *visual block* mode
Here you can visually select text and do manipulations on the visually selected text.

There is an idea behind all of that or let's say a philosophy. Once you have understood how it works, you won't need to memorize what may initially seem like cryptic key sequences, but you are able to create your own ones.

There is one more thing to mention to understand better what it is about Vim. It is based on the line editor `ex`, which itself is an evolution of `ed` as part of any Unix system. So, using Vim allows always to enter `ex` commands. I will not always indicate which is which. But to know that you have two editors in one and you can profit from both, results at the end in the enormous power of Vim. The power to do things in a very efficient way and with a minimum of key strokes. Please, don't misunderstand this as a religious statement, I'm not a combatant of the holy editor war between Emacs and Vim. At the end editors are used to edit text and your efficiency depends mainly on your specific editor skills, independent which one you prefer.

A brief note before we continue, if there are commands, which start with a colon `:`, they are `ex` commands and they have to be executed by hitting `↵`. Many Vim commands work without typing `↵`.

How to use this YaVI

It's not a step-by-step guide, but rather a paper to check quickly for things, which might be forgotten. If you are not experienced with Vim at all, you should read all of it before entering Vim on your machine, otherwise you struggle to enter or exit the program.

Vim itself has a powerful help integrated, which can be entered and asked any time you use Vim. Typing `:` `h` `<key word>` will split the screen and show information about your `<key word>`. Leave the help window with `:` `q`.

Meaning of the key symbols

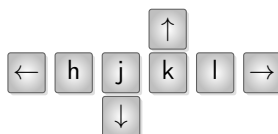
key	meaning	key	meaning
	enter/return key		shift key
	key x		shift key and x at the same time
	tabulator key		short for the same as above
	escape key		control key
	backspace		page down
	page up		

2 In motion

2.1 Jumping around

There are so many ways to navigate to a specific section of the text. Let's start with some basic movements while you are in `normal` mode.

- Type  to move the cursor one position to the left as long as you are not at the left most position on the line.
- Type  or  or  to move the cursor to the next line as long as you are not at the last line of your text.  and  in addition move the cursor to the first character on the line, whereas  keeps the column, when possible.
- Type  or  to move the cursor to the previous line as long as you are not at the first line of your text.  in addition moves the cursor to the first character on the line, similar to the previous point.
- Type  to move the cursor one position to the right as long as you are not at the right most position on the line.



These keys are adjacent to each other on your keyboard and the `j` has a tactile marker for the index finger of your right hand. In other words, the index finger moves one line forward, the middle finger one line above and the ring finger one position to the right. To move one position to the left, the index finger has to stretch one key to the left to reach the `h`.

All these movements can be augmented with a number.

1 5 |

This will move the cursor fifteen positions to the right. Moreover you can move your cursor by portions of text. These portions can be words, sentences, paragraphs or the whole text.

- Type the Vim command **gg** or the ex command **:1<** to move the cursor to the top of the buffer.
- Type the Vim command **↑+g** or the ex command **:\$<** to move the cursor to the end of the buffer.
- Type **w** or **↑+w** to move the cursor forward word by word till the end of the buffer. You will notice that a word in Vim has a special meaning. Try yourself to check this out. Type **b** or **↑+b** to move backwards. Type **e** or **↑+e** to move to the end of a word.
- Type **(** or **)** to move the cursor forward or backward sentence by sentence. You will notice that a sentence in Vim has a special meaning. Try yourself to check this out.
- Type **{** or **}** to move the cursor forward or backward paragraph by paragraph. A paragraph is surrounded by blank lines.
- Type the Vim command **5gg** or **5↑+g** to move the cursor to line number 5. There is another way to go to line number 5 by entering an ex command: **:5<**

There are also several movements on a single line:

- Type **0** to move the cursor to the start of the line and check for the difference to **^**.
- Type **\$** to move the cursor to the end of the line.
- Type **^** to move the cursor to the start of the line, but ignore any white space at the left and place the cursor on the first character.

- Type **f** **x** for find x to move the cursor to the next occurrence of letter x on the line. Repeat it with **;** in forward direction or **,** in backward direction.
- Type **shift** + **f** **x** to move the cursor to the previous occurrence of letter x on the line. Repeat it with **,** in backward direction or **;** in forward direction.
- Type **t** **)** to move the cursor just before the next occurrence of bracket **)** on the line. Repeat it with **;** in forward direction or **,** in backward direction.
- Type **shift** + **t** **(** to move the cursor just before the previous occurrence of bracket **(** on the line. Repeat it with **,** in backward direction or **;** in forward direction.

When ever useful, you can set marks inside your long text and jump from mark to mark. Vim will give you no feedback, when setting a mark nor any indication inside your text.

- Type **m** **a** to set a mark named a.
- Type **'** **a** to jump to the first character in the line where the mark named a is placed.
- Type **`** **a** to jump exactly to the position where the mark is placed.
- Type **:** **m** **a** **r** **k** **s** **<↵** to get a list of all marks below the status line. Note that there are a lot of other marks, which are set by Vim.

Lower case letters for marks are limited to the file itself, so you can have a mark a in several files. In contrast marks with a capital letter refer to a file and hence can only be used once. I have set a mark V in my `~/ .vimrc` file to open it, whenever I need it with only two key strokes **'** **V**.

Motion summary

keys	motion
h or ←	cursor to the left
j or ↓ or ↵ or +	cursor down
k or ↑ or -	cursor up
l or →	cursor to the right
15j or :+15↵	cursor 15 lines down
gg or :1↵	to top of the buffer
G or :\$↵	to bottom of the buffer
w or W	one word to the right
b or B	one word to the left
e or E	to the end of one word to the right
(or)	one sentence to the right or left
{ or }	one paragraph to the right or left
:5↵ or 5gg or 5G	go to line 5
0 or ^	go to first column in line
\$	go to last column in line
fx or Fx	go on next or previous occurrence of x in line
tx or Tx	go to next or previous occurrence of x in line
mx	set mark x
'x or `x	go to mark x

For more information type **:h motion**

2.2 Scrolling around

The commands to jump around not necessarily scroll the screen, if you have reached your destination. But if you want to examine your buffer there are several movements of the screen in the same way as you use other editors.

- Type **ctrl+f** for forward or  to scroll the screen one page forward. The last two lines on the screen will be the first afterwards.
- Type **ctrl+b** for backward or  to scroll the screen one page backward.
- Type **ctrl+d** for down to scroll the screen half a page forward. The last two lines on the screen will be the first afterwards.
- Type **ctrl+u** for up to scroll the screen half a page backward.
- Type **ctrl+y** scroll the screen one line down until the top of the buffer is reached on top of the window.

Note: There is an option in the Vim settings, which keeps a defined number of lines on top or bottom of your screen, refer to option `scrolloff`. I have set it to 5, which starts scrolling when I reach the top or bottom of my screen, so that I see always 5 more lines on the top or at the bottom.

Example: Assume your buffer has a lot of lines and you are at the top on line 1. The window you use displays 12 lines of your buffer at once. If your `scrolloff=5` and you type `ctrl+f` your screen will look like this:

```
1 Ich nehme den Begriff der Lebensweisheit hier gänzlich im
1 immanenten Sinne, nämlich in dem der Kunst, das Leben möglichst
2 angenehm und glücklich durchzuführen, die Anleitung zu welcher
3 auch Eudämonologie genannt werden könnte: sie wäre demnach die
4 Anweisung zu einem glücklichen Dasein. Dieses nun wieder ließe
5 sich allenfalls definieren als ein solches, welches, rein
6 objektiv betrachtet, oder vielmehr (da es hier auf ein
7 subjektives Urteil ankommt) bei kalter und reiflicher
8 Überlegung, dem Nichtsein entschieden vorzuziehen wäre. Aus
9 diesem Begriffe desselben folgt, daß wir daran hängen, seiner
10 selbst wegen, nicht aber bloß aus Furcht vor dem Tode; und
11 hieraus wieder, daß wir es von endloser Dauer sehen möchten.
```

scrolling_example

1,1

```
5 selbst wegen, nicht aber bloß aus Furcht vor dem Tode; und
4 hieraus wieder, daß wir es von endloser Dauer sehen möchten.
3 Ob nun das menschliche Leben dem Begriff eines solchen Daseins
2 entspreche, oder auch nur entsprechen könne, ist eine Frage,
1 welche bekanntlich meine Philosophie verneint; während die
16 Eudämonologie die Bejahung derselben voraussetzt. Diese näm
1 beruht eben auf dem angeborenen Irrtum, dessen Rüge das 49.
2 Kapitel im 2. Bande meines Hauptwerkes eröffnet. Um eine sol
3 dennoch ausarbeiten zu können, habe ich daher gänzlich abgehen
4 müssen von dem höheren, metaphysisch-ethischen Standpunkte, zu
5 welchem meine eigentliche Philosophie hinleitet. Folglich ber
6 die ganze hier zu gebende Auseinandersetzung gewissermaßen auf
```

scrolling_example

16,1

Scrolling cursor dependent

I often reach some part of the text and want to continue writing there and the cursor is already at the desired position. The only inconvenient thing is that I'm at bottom of the window and would like to see more below or I'm on the top of the window and want to see more from the lines above.

Vim has some helpful key strokes to adapt the screen relative to the cursor position.

- Type `zt` or `z↵` and move the line with the cursor to the top. If you have set `scrolloff=5`, 5 lines are left above. Check for the difference.
- Type `zz` and move the line with the cursor to the middle. Note: this works, if you are below of the middle and is useful, if you are more or at the end of the buffer.
- Type `zb` or `z-` and move the line with the cursor to the bottom. If you have set `scrolloff=5`, 5 lines are left below. Check for the difference.

Scroll summary

keys	motion
<code>ctrl+f</code> or <code>↓</code>	scroll forward one screen
<code>ctrl+b</code> or <code>↑</code>	scroll backward one screen
<code>ctrl+d</code>	scroll down half a screen
<code>ctrl+u</code>	scroll up half a screen
<code>ctrl+y</code>	scroll one line down
<code>ctrl+e</code>	scroll one line up
<code>zt</code> or <code>z↵</code>	move line to the top
<code>zb</code> or <code>z-</code>	move line to the bottom
<code>zz</code>	move line to the middle

For more information type **`:h scrolling`**

3 Working with text

As you can imagine, there are many ways to switch to `insert` mode. You are in `normal` mode and type one of the following letters to get to `insert` mode.

- Type `i` for insert and start typing at left of the cursor position. The look and feel depends a bit of the cursor layout. I have a vertical line cursor in `insert |` mode, but you also can have a block cursor `█`.
- Type `↑` + `i` for insert and start typing at the begin of a line. This is useful, if you are at any position on the line and want to add something at the beginning.
- Type `a` for append and start typing after the cursor position.
- Type `↑` + `a` for append and start typing after the end of a line. This is useful, if you are at any position on the line and want to add something at the end.
- Type `o` to get a new line below and you can start writing on this new line. This works from any position on the line you are.
- Type `5` `o` to get a new line below and you can start writing on this new line. When leaving `insert` mode the text will appear on 5 lines. This works from any position on the line you are.
- Type `↑` + `o` to get a new line above and you can start writing on this new line. This works from any position on the line you are.
- Type `s` for substitute and remove the letter under the cursor and continue writing.
- Type `5` `s` for substitute and remove five letters from the cursor onwards and continue writing.
- Type `↑` + `s` for substitute and remove the line and continue writing.

- Type **c**<motion> for change the text depending on the motion and continue writing.

A motion can be anything as described in section 2.1 on page 9.

c w will remove the word you are on and lets you keep writing. Imagine what **5 c w** will do or **5 w c**.

- Type **↑+c** for change and remove the text from the cursor to the end of the line and start writing or type **c c** for change and remove the line and continue writing. This is equivalent to **↑+s**.

To get out of the insert mode hit **↵**. Type **u** to undo your last action. Type **u** as long as your first action is reset. To go forward again type **ctrl+r** and re-do.

Insert summary

keys	inserting
i	insert at cursor position
↑+i	insert at begin of line
a	append after cursor position
↑+a	append at end of line
cte	removes all till e on the line and lets you keep writing
↑+c	change from cursor till end of line
o	open new line below and keep writing
5o	open a new line below and keep writing. When exiting the insert mode the text will appear on 5 lines
↑+o	open new line above and keep writing
s	delete character under the cursor and keep writing
↑+s	substitute line and keep writing
↵	escape from insert mode
u	undo the last action
ctrl+r	redo the last action

For more information type **:h inserting**

3.1 Deleting text

The scheme for deleting text is again the same as for other commands. You can delete characters, words, sentences, paragraphs, text between marks or between search patterns. You can delete all from the cursor to the bottom or

to the top or all and much more. All needs only a few key strokes and some knowledge about the philosophy of Vim.

The most used key when typing text in insert mode is backspace, but Vim allows more to delete. Type `ctrl+w` to delete the previous word or `ctrl+u` to delete the current line while staying in insert mode.

So let's have a look to my short summary, which gives you an idea how to use Vim for any type of deletion.

Delete summary normal mode

command	description
<code>x</code>	delete character under cursor
<code>4x</code>	delete 4 characters from cursor
<code>dw</code>	delete word forward
<code>db</code>	delete word backward
<code>d3w</code>	delete three words
<code>3dw</code>	delete three times a word
<code>3d3w</code>	delete three times three words
<code>d\$</code> or <code>^+d</code>	delete till end of line
<code>d0</code>	delete till start of line
<code>dd</code>	delete current line
<code>4dd</code>	delete current line and three more
<code>gagdG</code> or <code>Gdgg</code>	delete the buffer

Delete summary command mode

command	description
<code>d/pat↵</code>	delete current line till line with pat
<code>:1,5d↵</code>	delete lines 1 till 5 inclusive
<code>:1,\$d↵</code>	delete the buffer
<code>:'c,'g d↵</code>	delete lines from mark 'c till mark 'g inclusive

Delete summary insert mode

command	description
<code>ctrl+h</code> or <code>←</code>	delete previous character
<code>ctrl+w</code>	delete previous word
<code>ctrl+u</code>	delete current line

For more information type **:h delete**

3.2 Formatting text

Some programming languages rely on indentation, like Python or YAML. Vim allows you to choose between tabs or spaces. To customize this to your preference use the following parameters.

Indentation options summary

option	meaning
<code>:set tabstop=3</code>	when typing <code>→</code> a 3 spaces distance is used
<code>:set shiftwidth=3</code>	when typing <code>>↵</code> the line is indented with a distance of three spaces. It is recommended to have the same value for <code>tabstop</code>
<code>:set expandtab</code>	puts spaces, if <code>→</code> is typed
<code>:set noexpandtab</code>	puts a tab, if <code>→</code> is typed
<code>:set softtabstop=3</code>	defines the space for a tab in insert mode. Can you imagine why again 3?
<code>:set autoindent</code>	Vim indents the same as the line above, when you write on a new line
<code>:set noautoindent</code>	switch off the feature <code>autoindent</code>

You can make the invisible characters visible with `:set list`. Depending on your configuration you see tabulators, spaces and end of line symbols. Check how invisible characters are displayed with `:set listchars`

```
24  █some line of text
    table_example.md 24,3-7
:set list
```

```
24 → →some_line_of_text$
    table_example.md 24,3-7
```

L^AT_EX has a powerful formatting engine and you don't have to take much care in your source code about line lengths and so on. I use a for me convenient `textwidth` of 80 characters in my terminal.

When I'm typing and I reach the right end of the line, Vim inserts a `linebreak` and I can continue writing on the new line.

Line wrap summary

option	meaning
:set wrap	there will be no text beyond the window width
:set nowrap	there will may be text beyond the window width, if textwidth is larger than the current width of your window
:set textwidth=80	Vim inserts a line break after 80 characters
ggq or gww	inserts line breaks depending on textwidth for the current line

Autoformat program code

With very few key strokes you can properly indent ordinary code. Open your C-file and adapt all indentation automatically by typing

```
gg=G
```

gg puts to the first line of the buffer, = does the magic until end of the buffer G.

3.3 Moving text

From many other applications you are familiar with the key stroke combination **ctrl**+**c** and **ctrl**+**v** for cut and paste. Prerequisite is to select some portion of text beforehand.

Vim offers some interesting other key stroke combinations to move portions of text from one place to another. Let's start with a mistyped word like Linxu.

Place the cursor over the x and type **x** **p**. The result is Linux.

Imagine the following disordered lines:

3		column 1		column 2		column 3	
2		:-----		:-----:		-----	
1		bla 1		bla 2		bla 3	
24		blc 1		b ^l c 2		blc 3	
1		blb 1		blb 2		blb 3	
2		bld 1		bld 2		bld 3	

table_example.md24,15

Place the cursor somewhere in line 24 and type **d** **d** **p**. The result is:

21	column 1	column 2	column 3
1	:-----	:-----:	-----
2	bla 1	bla 2	bla 3
3	blb 1	blb 2	blb 3
4	blc 1	blc 2	blc 3
5	bld 1	bld 2	bld 3

table_example.md	21, 24
------------------	--------

One lesson from these examples is that deleted text isn't lost, but moved to some register in Vim and can be retrieved later on.

Now let's do an ordinary copy&paste text movement or in Vim-jargon a text movement with `yank&put`.

Step 1: Move the cursor somewhere inside a paragraph.

Remember a paragraph has empty lines around.

Step 2: Type `y i p` to yank the complete paragraph.

Step 3: Type `G p` to put the yanked paragraph at the end of the buffer or

Step 4: Type `g g P` to put the yanked paragraph at the top of the buffer

You can move you cursor where you want and put the yanked stuff after the cursor with `p` and before the cursor with `P`. It's also possible to put the stuff multiple times, if you add a number like this `8 p`.

Moving text with precision

Now we remember that Vim has the editor `ex` inside. I'll show you how to move text with `ex`-commands. Note that the following commands work always on whole lines.

The syntax to move lines consists of the range of lines you want to move and the destination where to put the lines.

```
: [range]m {destination}
```

As you can see, the `range` is optional and when omitted valid for the current line. `m` stands for move, if you replace this with `co` it copies. Yes, move can be abbreviated with `m`, but copy with `co`. The `destination` is a line address where to put the lines.

The following examples will clarify this.

Move summary

command	description
<code>:m 0</code>	move the current line to the top of the buffer
<code>:m 1</code>	move the current line to line 1
<code>:m \$</code>	move the current line to bottom of the buffer
<code>:m -2</code>	move the current line 2 lines above
<code>:m +1</code>	move the current line 1 line below
<code>:10,15 m 20</code>	move the line 10 to 15 to line 20

For more information type **:h :m**

These are some examples with explicit line numbers, but it is also possible to base on search results. Look to the following interesting examples.

Move and copy summary

command	description
<code>:'<,'> m 0</code>	this appears, if you are in visual mode and you type <code>:. </code> Then the selected text moves to the top of your buffer
<code>:g/section/ m \$</code>	this globally searches for the pattern section in each line and puts all lines, which match to the bottom of your buffer
<code>:'a,'b m \$</code>	this moves the lines between mark 'a and 'b to the bottom of your buffer
<code>:?<pat1>?,/<pat2>/co \$</code>	this copies the lines between the backward search result <pat1> to forward search result <pat2> at the bottom of the buffer.

For more information type **:h :co**

The last line in the box above shows the power of Vim . It especially works if you are inside a paragraph and you want to match the paragraph boundaries. If you want to stay inside the boundaries you can specify it like the following:

```
:?<pat1>?+1,/<pat2>/-1 co $
```

Other ranges could be <search>, <mark> or <mark>, <line number> or anything else. You may not use such rather complicated movements, but imagine a long document and you want to rearrange the sections. You may want to use a macro to it for you on many sections with help of these features.

One last, but rather useless command inverts the order of lines in your buffer, if you type

```
:g/^/ m $.
```

This command globally `g` matches any line, because all lines have a start of line `^`. Each matched line is put to the end, line by line, which reverses the order.

4 Hide and seek

Searching for patterns and replacing them is one of the most powerful features of Vim and more powerful than most of the other implementations around. It is absolutely worth to learn how regular expressions work to be able to manipulate your text most efficiently.

Make your search results visible with following useful highlighting options:

Option 1 `:set hlsearch`

Stands for highlight search and highlights all matches in the buffer

Option 2 `:set incsearch`

Stands for incremental search and immediately starts highlighting while typing your search pattern

As usual the commands `:set nohlsearch` and `:set noincsearch` deactivate the options. You can also use `:set hlsearch!` and `:set incsearch!`.

Before we start I like to mention some general things, to give an overview on the features and restrictions, when you work with regular expressions in Vim.

Point 1: Regular expressions are executed line by line. You have to do extra work, if they shall span over line breaks.

Example: `\_` within a regular expression stands for line break.

Point 2: Some characters have special functions and have to be masked with `\`, if you want them literally.

Example: `$` function is end of line, literal is `\$`

Point 3: Some literals get special functions, if they are masked with the `\`

Example: literal is `d`, `\d` stands for the collection of digits.

Point 4: You can use anchors to restrict your regular expression to special positions. Positions could be word, line or buffer boundaries, as well as marks. For more information type `h: pattern-atoms`

Example: `/\<ment` matches `mentor`, but not `replacement`

Point 5: You can specify the range where your replacement shall be executed. This range may be a visual selection, a line, some lines, the buffer or many buffers.

Example: `:42,$ s/yes/no/g` replaces `yes` by `no` in the current buffer, starting from line 42 till end of the buffer `$` and does it globally `g` for all occurrences on each line of the range.

Point 6: You can store search patterns for later use. This is interesting, when you do replacements.

Example: `:%s/\(ements\) \ (replac\) /\2\1/` would substitute `ementsreplac` by `replacements`.

Point 7: You can do conditional searches and replacements with the function of look ahead and look behind.

Example 1: `/reg\(ex\) \@=` matches `reg` in `regex`

Example 2: `/reg\(ex\) \@!` matches `reg` if `ex` doesn't follow.

For more information type **`:h pattern-multi-items`**

Some basics

Let's start with some basic search commands and look for replacements later.

- Type `*` or `#` to search forward or backward for the word under the cursor.
- Type `/`<pattern> to search forward or `?`<pattern> to search backward for your <pattern> in the text.
- Type `n` or `N` to jump to the next or previous occurrence. The direction to the next occurrence depends on how you started to search. You can try yourself.
- Type `f` `x` to find the next letter `x` on the line.
Continue searching for the next `x` by typing `;` or `,` for the previous one.
Refer also to section 2.1 on page 11.
- Type `↑` `f` `x` to find the previous letter `x` on the line.
Continue searching for the previous `x` by typing `;` or `,` for the next one.

- Type `t)` to place the cursor right before the next `)` on the line.
Continue searching for the next `)` by typing `;` or `,` for the previous one.
I used the bracket in this example intentionally.
- Type `↑+t(` to find the previous letter `(` on the line.
Continue searching for the previous `(` by typing `;` or `,` for the next one.
- Type `:g/<pattern>/p` to print all lines which contain the pattern. The matching lines appears below the status line. If you have long structured text and you want to know what sections are in, this command can give you the answer. `:g/section/p`
: enters command mode, g searches globally in each line of the current buffer for the pattern section between the delimiters / and / and puts p the result below the status line as shown below.

```

33 Ich nehme den Begriff der Lebensweisheit hier gänzlich im
1  immanenten Sinne, nämlich in dem der Kunst, das Leben möglichst
2  angenehm und glücklich durchzuführen, die Anleitung zu welcher
3  auch Eudämonologie genannt werden könnte: sie wäre demnach die
4  Anweisung zu einem glücklichen Dasein. Dieses nun wieder ließe
5  sich allenfalls definieren als ein solches, welches, rein

```

philosophers.tex

33,1

```

:g/section/p
1  \section{Schopenhauer}
32 \section{Kant}
32 \section{Hegel}

```

As you can imagine there must be more than searching for literals. How can I search for `\` or `*` in the text, if these characters seem to have a special meaning?

List of special characters

Following list of special characters can't be used as literals for a search pattern. They are only a few. To get rid of its special function, they have to be masked with backslash `\`.

Special characters

char	meaning
\$	anchor at end of line
\\$	literal \$
^	anchor at start of line
\^	literal ^, circumflex
*	multiplier, 0 or multiple times
*	literal *, star
.	wildcard, except new line
\.	literal ., dot
_	add end of line character
	alternative
\	literal , pipe symbol

For more information type **:h pattern-atoms**

Let's look at some examples for these special characters.

How do I match a string at the line end?

```
/Style$
```

```
1 Flash Sale | Level ^ Your Style
3 Package      Value      Included      *
4 Starter      Budget-friendly . Essential kit **
5 Premium      Performance ^ Advanced tools **
6 Ultimate     Total Control  * All-access pass **
8 Don't miss out | Act now ^ and save big $ .
```

The / indicates searching, Style is the pattern you search for and \$ is the anchor of the end of the line.

How do I match any \$?

```
/\$
```

```
1 Flash Sale | Level ^ Your Style
3 Package      Value      Included      *
4 Starter      Budget-friendly . Essential kit **
5 Premium      Performance ^ Advanced tools **
6 Ultimate     Total Control  * All-access pass **
8 Don't miss out | Act now ^ and save big $ .
```

The / indicates searching, \ masks the function of the end of the line and makes \$ literal.

How do I match the P at begin of the line?

`/^P`

```
1 Flash Sale | Level ^ Your Style
3 Package      Value                Included      *
4 Starter      Budget-friendly .    Essential kit **
5 Premium      Performance          ^            Advanced tools **
6 Ultimate     Total Control        *            All-access pass **
8 Don't miss out | Act now ^ and save big $ .
```

The `/` indicates searching, `^` is the anchor of the start of the line and `P` is the pattern you search for.

How do I match two or more spaces?

`/_ _ _ *`

```
1 Flash Sale | Level ^ Your Style
3 Package      Value                Included      *
4 Starter      Budget-friendly .    Essential kit **
5 Premium      Performance          ^            Advanced tools **
6 Ultimate     Total Control        *            All-access pass **
8 Don't miss out | Act now ^ and save big $ .
```

The `/` indicates searching, `_ _` are at least the two spaces you search for and `_ *` multiplies one space by 0 or more.

How do I match Style followed by Package

`/Style\n\nPackage or`

`/Style\_. *Package`

```
1 Flash Sale | Level ^ Your Style
2
3 Package      Value                Included      *
4 Starter      Budget-friendly .    Essential kit **
5 Premium      Performance          ^            Advanced tools **
6 Ultimate     Total Control        *            All-access pass **
8 Don't miss out | Act now ^ and save big $ .
```

The `/` indicates searching, `Style` is part of the pattern you search for, `\n\n` stands for two line breaks and `Package` is the rest of you search pattern.

The `/` indicates searching, `Style` is part of the pattern you search for,

_.* stands for any characters including the end of the line and Package is the rest of you search pattern.

How do I match Pa or Pr?

```
/Pa\|Pr or  
/P[ar]
```

```
1 Flash Sale | Level ^ Your Style  
  
3 Package      Value      Included      *  
4 Starter      Budget-friendly . Essential kit  **  
5 Premium      Performance ^ Advanced tools **  
6 Ultimate      Total Control * All-access pass **  
  
8 Don't miss out | Act now ^ and save big $ .
```

The / indicates searching, Pa is one part of the pattern you search for, \| indicates the alternative pattern Pr.

The / indicates searching, P is part of the pattern you search for, [] forms a groups and a or r are part of the pattern.

How do I match a string, starting with P until next e?

```
/P.\{-}e
```

```
1 Flash Sale | Level ^ Your Style  
  
3 Package      Value      Included      *  
4 Starter      Budget-friendly . Essential kit  **  
5 Premium      Performance ^ Advanced tools **  
6 Ultimate      Total Control * All-access pass **  
  
8 Don't miss out | Act now ^ and save big $ .
```

The / indicates searching, P is one part of the pattern you search for, . followed by \{-} matches any character, but as less as possible and e is the last part of your pattern.

How do I match Total, if followed by Control?

```
/Total\ze Control
```

```
1 Flash Sale | Level ^ Your Style  
  
3 Package      Value      Included      *  
4 Starter      Budget-friendly . Essential kit  **  
5 Premium      Performance ^ Advanced tools **  
6 Ultimate      Total Control * All-access pass **  
  
8 Don't miss out | Act now ^ and save big $ .
```

The / indicates searching, Total is one part of the pattern you search for, \ze sets the end of the match and _Control is the other part of the pattern, but not matched.

How do I match Control, if preceded by Total?

/Total \zsControl

```
1 Flash Sale | Level ^ Your Style
3 Package      Value                Included      *
4 Starter      Budget-friendly .    Essential kit **
5 Premium      Performance          ^    Advanced tools **
6 Ultimate     Total Control      *    All-access pass **
8 Don't miss out | Act now ^ and save big $ .
```

The / indicates searching, Total_ is one part of the pattern, but not matched, \zs sets the start of the match and Control is the other part of the pattern you search for.

How do I match form, if preceded by Per and followed by ance?

/Per\zsform\zance

```
1 Flash Sale | Level ^ Your Style
3 Package      Value                Included      *
4 Starter      Budget-friendly .    Essential kit **
5 Premium      Performance          ^    Advanced tools **
6 Ultimate     Total Control      *    All-access pass **
8 Don't miss out | Act now ^ and save big $ .
```

The / indicates searching, Per is one part of the pattern, but not matched, \zs sets the start of the match, form is the pattern you are searching for, \ze sets the end of the match and ance is the part of the pattern that follows, but it is not matched.

How do I match s, if not preceded by a and followed by s?

/\ (a\)\@<!s\ (s\)\@=

```
1 Flash Sale | Level ^ Your Style
3 Package      Value                Included      *
4 Starter      Budget-friendly .    Essential kit **
5 Premium      Performance          ^    Advanced tools **
6 Ultimate     Total Control      *    All-access pass **
8 Don't miss s out | Act now ^ and save big $ .
```

The `/` indicates searching, `\(` and `\)` build a group for the preceding pattern `a`. `\@<!` is negative and inverts the group with `a` inside, meaning everything except `a`. Then follows the pattern `s` you search for and again a group with `s` inside, which is not matched. `\@=` is positive, the group shall follow.

One note to the `.` dot. It is denoted as wildcard, but there is one character, which is not matched, the end of line character. If you want to match patterns, which span beyond the end of line, you have to mention this explicitly.

Since there are many more special characters needed, but the character set is limited, many literal characters get a special function, if a backslash `\` is added.

More special characters

char	meaning
<code>\e</code>	escape ⌘
<code>\t</code>	tabulator →
<code>\r</code>	carriage return ↵
<code>\b</code>	backspace ←
<code>\+</code>	multiplier, at least one
<code>\(...\)</code>	make group
<code>\1...\9</code>	refer up to nine groups
<code>\=</code>	optional character
<code>\<</code>	anchor start of word
<code>\></code>	anchor end of word
<code>\(...\)\@<=</code>	positive look behind
<code>\(...\)\@=</code>	positive look ahead
<code>\(...\)\@<!</code>	negative look behind
<code>\(...\)\@!</code>	negative look ahead
<code>\{x,y\}</code>	<code>x</code> to <code>y</code> counts

Look to some more examples with more special characters.

How do I match words ending with `le`?

```
/\w\{-\}le\>
```

```
1 Flash Sale | Level ^ Your Style
3 Package      Value      Included      *
4 Starter      Budget-friendly . Essential kit **
5 Premium      Performance ^ Advanced tools **
6 Ultimate      Total Control * All-access pass **
8 Don't miss out | Act now ^ and save big $ .
```

The `/` indicates searching, `\w` stands for word characters `\{-}` stands for as less as possible word characters, `le` is part of the pattern you search for and `\>` is the anchor of the end of a word.

There is also the possibility to express ranges of literals or character classes. This is a way to keep your search pattern string short and clear.

Even more special characters

char	meaning
<code>[abc]</code>	matches a or b or c
<code>^x</code>	matches anything except x
<code>\a</code>	alphabet, short for <code>[a-zA-Z]</code>
<code>\d</code>	digits, short for <code>[0-9]</code>
<code>\D</code>	non digits, short for <code>[^0-9]</code>
<code>\o</code>	octal digits, short for <code>[0-7]</code>
<code>\O</code>	non octal digits, short for <code>[^0-7]</code>
<code>\x</code>	hex digits, short for <code>[0-9a-fA-F]</code>
<code>\X</code>	non hex digits, short for <code>[^0-9a-fA-F]</code>
<code>\w</code>	word characters, short for <code>[a-zA-Z0-9_]</code>
<code>\W</code>	non word characters, short for <code>[^a-zA-Z0-9_]</code>
<code>\s</code>	white space, short for <code>[\ _]</code>
<code>\S</code>	non white space, short for <code>[^\ _]</code>
<code>\l</code>	lowercase, short for <code>[a-z]</code>
<code>\L</code>	non lowercase, short for <code>[^a-z]</code>
<code>\u</code>	uppercase, short for <code>[A-Z]</code>
<code>\U</code>	non uppercase, short for <code>[^A-Z]</code>

For more information type `:h /\s`

The list of POSIX character classes like `[:digit:]` is not part of this YaVI.

4.1 Vim is case sensitive

If you haven't had contact to regular expressions in the past, this section might be a bit confusing to you. Anyhow, you will profit – I'm pretty sure.

It is impossible to imagine what you want to search for in the future or moreover to replace. So, my idea is to give you some basics, which can be assembled and combined to work for your future search and replace adventure.

If you had already contact to regular expressions, e.g. in PERL¹, be careful, because it's not all the same and compatible with each other. I present in this YaVI only the regex, which are valid for Vim.

¹Practical Extraction and Reporting Language

As common use in Unix and Linux Vim respects the difference between `pattern`, `Pattern` and `PATTERN`. If you have code, which is a bit indifferent with respect to upper- and lowercase letters, like SQL, you may want to find any appearance of your search pattern.

```
:set ignorecase
```

will do it for you. To get back to a strict search function, type:

```
:set noignorecase
```

But Vim provides also a quite comfortable solution to have both more or less in one. If you type:

```
:set ignorecase smartcase
```

This matches all, if you only type lowercase letters, but is strict as soon as you have any uppercase letter in your `pattern`. If you don't want to modify your general setting, you can force a single search case sensitive or case insensitive independent of your setting.

```
/\cpattern will find pattern, patTern, PATTERN, ...
```

```
/\CpatTern will find patTern only
```

4.2 Replacing

Replacing is preceded by searching. You have to define a pattern, which then will be replaced.

```
:s/search pattern/replacement/
```

The line above enters the command mode of the editor `ex` with the `:`, then follows the command `s` for substitute, followed by three delimiters² `/` and in between what shall be replaced by what.

The command written like this is limited to the current line and replaces only the first occurrence of `search pattern` in this line, if any. The command becomes more interesting and useful, if it works on multiple lines. To reach this you can define a range, which precedes the command.

```
: [range] s/search pattern/replacement/
```

There is a big variety of ranges to be defined. Chose what could fit for your replacement adventure from the table below.

²Any delimiter out of these can be used: `+` `!` `,` `;` and `%`

Ranges summary

range	description
%	use the whole buffer as range, very often used
1,\$	same as %, from line 1 till end of buffer
15	on line 15
15,+3	on line 15 and following 3 lines
'h,/pattern/	from mark 'h till position of next occurrence of pattern

For more information type **:h range**

If you want to control your substitutions, you can append several options to your search command.

```
:s/search pattern/replacement/options
```

Option g: With this option all occurrences on the line are substituted. If not set, only the first occurrence is substituted.

Option c: The c stand for confirm and asks for each occurrence whether to substitute or not. You can answer with:

```
y  yes, substitute
n  no, skip
l  last substitution, then quit
q  quit
a  all, substitute all remaining occurrences
```

The option g and c can be combined.

5 Working with files

5.1 Opening files from the terminal

I often search for strings in files. I use `grep`¹ to do this for me. `grep` has some nice features to not only return the file name and line with the pattern, but it can ignore upper- and lowercase respectively and it can give the line number.

An example could be:

```
grep -i -r -n itemize *.tex
```

This command ignores (`-i`) the case, searches recursively (`-r`) also through all subdirectories and gives the line number (`-n`) of the search result. It searches for the pattern `itemize` in files with the ending `tex`.

Imagine following output on the console:

```
section_opening_files.tex:71:\begin{itemize}  
section_opening_files.tex:82:\end{itemize}
```

With this information I can open the file exactly at the position I want.

```
vim +71 section_opening_files.tex
```

And there are more ways to open a file in `vim`.

```
vim +/itemize section_opening_files.tex
```

This opens the file `section_opening_files.tex` at the first position where the search pattern `itemize` occurs.

Open files in window split mode

```
vim -o file_1.tex file_2.tex
```

¹`grep` stands for global regular expression print, which is derived from editor `ex`.
global/regular expression/print or short `g/re/p`

This command line option opens both files and displays them in two different windows of your terminal. The split is horizontally. File `file_1.tex` is on the top, `file_2.tex` is on the bottom. The window on the top is active.

```
vim -o file_1.tex file_2.tex
```

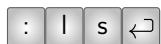
The capital letter `O` does the same, but with a vertical split of the screen.

How to work with windows like closing, switching and so on, is described in more detail in section 7.1 on page 45. Moreover there is the option `-p` to open files in so called tabs. This is not part of this YaVI. For information on this type `:h tab-page-intro`.

You can also open a bunch of files with following command:

```
vim *.tex
```

I use this quite often to open all my $\text{\TeX}$ -files in a project. With all files open in buffers, I can work on each file or on many files at the same time. Vim allows to do substitutions on one file, on the files visible in several windows or on all files opened in the memory respectively buffers. To switch from file to file you may first use the following command to list the files opened:



The output of `:ls` may look like this:

```
33 Ich nehme den Begriff der Lebensweisheit hier gänzlich im
1  immanenten Sinne, nämlich in dem der Kunst, das Leben möglichst
2  angenehm und glücklich durchzuführen, die Anleitung zu welcher
3  auch Eudämonologie genannt werden könnte: sie wäre demnach die
4  Anweisung zu einem glücklichen Dasein. Dieses nun wieder ließe
5  sich allenfalls definieren als ein solches, welches, rein

Schopenhauer.tex 33,1

:ls
1 # "Kant.tex" line 52
2 %a "Schopenhauer.tex" line 33
3 "Hegel.tex" line 34
```

You can read this output as follows: Currently three files are opened.

- file `Kant.tex` is loaded in buffer 1 and the cursor is in line 52
- file `Schopenhauer.tex` is loaded in buffer 2 and so on and the cursor is in line 33

- a indicates the active buffer, the file you see on the screen
- % indicates the active window, which helps only in window split mode
- # indicates the alternate buffer, which is the previously used buffer. You can switch between two buffers using the command `:e#`, but there are other ways to do so.

You can switch around in the buffers with:

`:bn`, which counts forward or

`:bp`, which counts backwards

and you can address the buffers directly with

`:b3`, which jumps to buffer three.

Sometimes, if you have opened many files at the same time, you want to get rid of one file in the list of buffers. It will not be deleted from your file system. You can do it with following command:

`:bd`

If you want another file to be removed from the list add the buffer number like this:

`:bd3`

If you enter `:ls` again, you will notice that the numbers of the remaining buffers are kept. So, they keep stable during the whole session.

5.2 Open files inside Vim

To open an existing file, when Vim is already started and you have a buffer on the screen, type `:e<filename>`

To open a new empty buffer type `:new` or `ctrl+w n`.

For more information type `:h new`

Vim has a file manager built in. If you type `:e.` a file manager is opened. Move inside with the usual motions and open a file under cursor by typing `<cr>`.

5.3 Start writing

Normally you are in normal mode, when you are opening a file. With the option `-c start` you'll get directly into insert mode.

```
vim -c start "+/markdown file" macros.tex
```

This line will open the file `macros.tex`, search for the pattern `markdown file`, position the cursor to the start of the line where this pattern appears first and put you in insert mode with the ex command `-c start`.

Buffers summary

command	description
<code>vim file</code>	opens file at first line
<code>vim + file</code>	opens file on last line
<code>vim +n file</code>	opens file at line <code>n</code>
<code>vim +/pattern file</code>	opens file at pattern
<code>vim -o file1 file2</code>	opens both files in horizontal split mode
<code>vim -O file1 file2</code>	opens both files in vertical split mode
<code>vim -d file1 file2</code>	opens both files in diff mode
	refer to section 7.2 on page 47

5.4 Closing files

Now you have changed a lot in your buffer or buffers and you want to leave. There are many ways to exit Vim. For the following commands I assume you have pressed `␣` and you are in normal mode.

- Type `:q!` for quit forced to discard any changes and close the buffer. If you have tried to create a new one, there will be none.
- Type `:q` for quit to close the buffer. You will get an error message, if you missed to save your changes. By the way you can see this in your status line. A `[+]` behind the buffer name indicates that there is unsaved stuff.
- Type `:qa` for quit all, if you have opened multiple buffers. Remember that you can check for all opened buffers with `:ls` or `:files` and you can see at the `+`, if there they have unsaved modifications.

- Type    for write quit to save your work and quit Vim. The same effect has  , one letter less to press.

6 Working with registers

The first thing you can do is to look at your registers. Type  and get a long list below the status line with all the registers and their contents, but what does this all mean?

Some registers are set by Vim others can explicitly set by you. For more information type **:h registers**.

6.1 Some types of registers

1. The unnamed register ""
Vim stores here any recent deletion you made.
2. 10 numbered registers "0 to "9
Vim stores the 10 recent deletions you made. Where deletion n+1 is older than n.
3. 26 named registers "a to "z or "A to "Z
Type "gyw to yank the current word and replace with register "g.
Type :reg g to check for the contents of register "g.
Type "gp to put the content of register "g somewhere.
Type "Gyw to yank and append the current word to register "g.
Type :reg g again to check for the contents of register "g. The second word should be appended to register "g.
Type qr to start recording a macro. All keystrokes are put into register "r. Quit recording with q.
Type :reg r to check for the contents of register "r.
Type @r to call the macro in register "r.
4. Three read-only registers ":", ".", "%
Type "%p to put the current file name.
Type "@: to execute the recent command entered on the command line. It works only, if you have a command line history enabled.
Type ".p to put the recent inserted text somewhere.
Type :reg . to check for the contents of register ".

5. Last search pattern register `/`
Type `:reg /` to check for the last used search pattern.
6. The expression register `=`. It's not a register, but evaluates expressions and can do calculations for you. Refer to section 8.2 on page 52.

As you can imagine most of the time you don't have to check for the contents of all these registers, but you will use them always. The next section will show an example for the explicit usage of named registers.

6.2 Using macros

Code for computers and its documentation often contains stupid repetitions line by line and the differences are sometimes one letter or a number, which counts. If you have to do changes on these very similar lines, you may not want to do it again and again. For this cases you can record your typing once and play it as often as you want.

Step 1: Type `qa` to start recording the macro with name a. Look to your status line, it's indicated there.

Step 2: Type a sequence of movements and commands.

Step 3: Quit recording with typing `q` again.

Step 4: You may check what you have recorded. It's stored in a register.

Type `:reg` and look for "a.

Step 5: Run the macro a by typing `@a`

Again, you can augment the execution with a number: `5@a` and the macro is executed five times.

Let's make an example. I use a table in a markdown file.

```
21 | column 1 | column 2 | column 3 |
1  | :----- | :-----: | ----- |
2  | bla 1   | bla 2   | bla 3   |
3  | blb 1   | blb 2   | blb 3   |
4  | blc 1   | blc 2   | blc 3   |
5  | bld 1   | bld 2   | bld 3   |
```

`table_example.md`

21, 24

I want to be the `blx` in UPPERCASE and be placed after the number.

Step 1: Type `q a` to start recording the macro with name `a`.

Step 2: Type `/ b l . ↵` to search for any type of `blx`.

Step 3: Type `g n` to select the next search result.

Step 4: Type `~` to switch case.

Step 5: Type `d w l p` to delete the word, move to the right and paste.

Step 6: Type `q` to stop recording.

Step 7: Type `1 2 @ a` to execute the macro 12 times.

The result should look like this:

```

5 | column 1 | column 2 | column 3 |
4 | :----- | :-----: | ----- |
3 | 1 BLA    | 2 BLA    | 3 BLA    |
2 | 1 BLB    | 2 BLB    | 3 BLB    |
1 | 1 BLC    | 2 BLC    | 3 BLC    |
26 | 1 BLD    | 2 BLD    | 3 BLd    |

```

`table_example.md`

26, 28

7 Working with windows

There are only a few use cases where I use the ability to split my terminal screen.

Use case 1: The text is too long and I want to see a part on the top and at the same time a part at the bottom.

Use case 2: I have two files and want to compare the contents to check for the differences.

Use case 3: I want to see what I yank from one buffer to the other.

Use case 4: I need help from Vim by typing `:h <unknown>`.
This splits the screen automatically and the focus is on the help.

7.1 Split the screen

You have already read in section 5.1 on page 35 that Vim may directly open files in window split mode. You can split your current screen any time by typing

`: s p l i t`

This can be repeated until you have many useless small pieces of your file in many windows.

`: v s p l i t`

does the same, but vertically. Both can be alternated to get a funny tiled screen. Type

`: q`

to close the active window. Note that this does not quit Vim as long as there are open windows. Moreover the buffers remain in the list of buffers and they are not removed.

My further examples are limited to one split, which means two windows on the screen. This may look like this:

1 Ich nehme den Begriff der Lebensweisheit hier gänzlich im
1 immanenten Sinne, nämlich in dem der Kunst, das Leben möglichst
2 angenehm und glücklich durchzuführen, die Anleitung zu welcher
3 auch Eudämonologie genannt werden könnte: sie wäre demnach die
4 Anweisung zu einem glücklichen Dasein. Dieses nun wieder ließe
5 sich allenfalls definieren als ein solches, welches, rein

Schopenhauer.tex1,1

33 Vervollständigen kann. Zwar hat auch Aristoteles dem 5. Kapi
1 des 1. Buches seiner Rhetorik eine kurze Eudämonologie
2 eingeflochten: sie ist jedoch sehr nüchtern ausgefallen. Benu
3 habe ich diese Vorgänger nicht; da Kompilieren nicht meine Sa
4 ist; und um so weniger, als durch dasselbe die Einheit der
5 Ansicht verloren geht, welche die Seele der Werke dieser Art

Schopenhauer.tex33,1

Type + + to switch the focus.

1 Ich nehme den Begriff der Lebensweisheit hier gänzlich im
1 immanenten Sinne, nämlich in dem der Kunst, das Leben möglichst
2 angenehm und glücklich durchzuführen, die Anleitung zu welcher
3 auch Eudämonologie genannt werden könnte: sie wäre demnach die
4 Anweisung zu einem glücklichen Dasein. Dieses nun wieder ließe
5 sich allenfalls definieren als ein solches, welches, rein

Schopenhauer.tex1,1

33 Vervollständigen kann. Zwar hat auch Aristoteles dem 5. Kapi
1 des 1. Buches seiner Rhetorik eine kurze Eudämonologie
2 eingeflochten: sie ist jedoch sehr nüchtern ausgefallen. Benu
3 habe ich diese Vorgänger nicht; da Kompilieren nicht meine Sa
4 ist; und um so weniger, als durch dasselbe die Einheit der
5 Ansicht verloren geht, welche die Seele der Werke dieser Art

Schopenhauer.tex33,1

Window commands

The commands to work with windows start always with +. This is rather easy to keep in mind – w for window.

Window commands summary

keys	effect
ctrl+w s or S or ctrl+s	split the screen horizontally
ctrl+w v or ctrl+v	split the screen vertically
ctrl+w ctrl+w	toggle the focus of the window
ctrl+w q or ctrl+q	close the current window
ctrl+w c	close the current window
:q or :close	close the current window
:only	make the current window visible only
ctrl+w <motion>	try yourself moving the focus
ctrl+w r or R or ctrl+r	rotate the windows
ctrl+w =	make the windows the same size
ctrl+w +	increase window height (horiz. split)
ctrl+w -	decrease window height (horiz. split)
ctrl+w >	increase window width (vert. split)
ctrl+w <	decrease window width (vert. split)

For more information type **:h windows**

7.2 Diff mode

Opening two files in vertical split mode allows to check for the differences in a very convenient way. You can do this manually or you can use the so called diff mode of Vim.

I use Git and from time to time I get a merge conflict, which has to be solved. I do this with Vim as diff tool for Git. The following commands open the two file in diff mode, which means in vertical split and with several options set to compare both files.

```
vim -d file_version.1 file_version.2
vimdiff file_version.1 file_version.2
```

If you prefer a horizontal split, add option `-o` and the window split is horizontal.

```
vim -d -o file_version.1 file_version.2
vimdiff -o file_version.1 file_version.2
```

The diff mode is not limited to two files, you can open many files. The limit is the screen size and your capacity to keep the overview.

If you have already opened two files, you can activate the diff mode afterwards.

```

1 Ich nehme den Begriff der Le      | 1 Ich nehme den Begriff der Le
1 immanenten Sinne, nämlich in    | 1 immanenten Sinne, nämlich in
2 angenehm und glücklich durc      | 2 angenehm und glücklich durc
3 auch Eudämonologie genannt w    | 3 auch Eudämonologie genannt w
4 Anweisung zu einem glücklich    | 4 Anweisung zu einem glücklich
5 sich allenfalls definieren a    | 5 sich allenfalls definieren a
file_version.1      1,1      file_version.2      1,1

```

Lines with differences are highlighted, depending on the color scheme and the difference in the line is also visible. If you want to align both files you can chose which version you want. The active window in the example above holds the buffer of `file_version.2`, which I want to be taken.

```
:diffput
```

will put the difference to the other buffer. As soon as there are no more differences, everything, which is equal is folded.

```

+1 +-- 6 lines:  Ich nehme den      | +1 +-- 6 lines:  Ich nehme den
~                                     | ~
~                                     | ~
~                                     | ~
~                                     | ~
~                                     | ~
file_version.1      1,1      file_version.2      1,1

```

diff mode summary

command	description
:diffthis	to be typed in each window, which shall be compared
:diffget	make the current line equal to the other buffer
:diffput	make the other buffer equal to the current line

For more information type **:h diff**

7.3 Commands for all windows

Imagine you have different files opened in different windows and you want to execute a command, which shall manipulate the text in the same way in all

windows.

Besides commands to synchronize the behaviour of the windows like

```
:windo set scrollbind  
:windo set nowrap
```

which helps to compare file contents, you can do replacements in all windows with only one command.

```
:windo s/<search>/<replace>/gc
```

What does it? It searches for the pattern <search> in all windows :windo and replaces it with <replace> globally g, which means all occurrences on a line and asks for confirmation c.

The same logic applies for buffers.

8 Features I love

8.1 Abbreviations

I have written this YaVI with help of $\text{\LaTeX}$ and abbreviations in Vim may reduce the number of key strokes. Ok, I can also use means of $\text{\LaTeX}$, but here I like to show how Vim can help typing less.

It's rather simple. Type

```
:ab <ABBREV> <LONG TEXT>↵
```

and Vim stores the <LONG TEXT> under abbreviation <ABBREV>.

I use quite often `\texttt{ }` to set text with teletype letters. My abbreviation was made like this

```
:ab tt \texttt{↵
```

Vim replaces `tt` when I'm in `insert` mode and type `tt` followed by a space. If you don't want the replacement, type `ctrl+v` after the abbreviation. To list all abbreviations type `:ab↵` and all abbreviations are listed below the status line.

8.2 Counting and calculating

Vim can count. If you have any number in your text and you want to increase or decrease it, you can do so by typing `ctrl+a` to add one or `ctrl+x` to subtract one. Typing `4 2 ctrl+a` adds 42 to your number.

I guess you won't need this feature very often, but it could serve sometime when you want to create a stupid numbered list. Neither $\text{\LaTeX}$ nor Markdown needs this kind of explicit numbering, but I'll describe it anyhow.

Let's record a macro that shall do the job. A number 0 shall already be there and we go to this number.

Step 1: Start recording the macro `n` by typing `q n`.

Step 2: Increase the number by typing `ctrl+a`.

Step 3: Go to the first column by typing `0`. This is important, if the numbers have more than one digit.

Step 4: Yank the number by typing `yw`.

Step 5: Add a new line by typing `o` and escape insert mode with `↵`.

Step 6: Put the number by typing `p`.

Step 7: Stop recording the macro by typing `q` again.

You can now create a list of 42 numbers by typing `42@n`. The macro continues, if you want more.

Vim can calculate. Imagine a table with some values and you want to summarize them or do some other calculations. Vim can do it for you.

Step 1: Go to insert mode and type `ctrl+r=`.

Step 2: You get a `"` at the cursor and a `=` below the status line.

Step 3: Now enter an expression like `33*42` and press `↵`

Step 4: the `"` is replaced by `1386`.

For more information type `:h "=`

8.3 Features for programmers

To keep track of a large number of lines of code Vim has some features to hide rather less important parts of your code. This feature is called *folding*. The letter, which is chosen for this is `z`. It is meant to remind you of a folded sheet of paper.

- Type `10zf` to fold the 10 following lines starting from the cursor.
- Type `zo` for open to unfold the fold

- Type `z a` to toggle the fold
- Type `z c` for close to close the fold
- Type `z d` for delete to remove the fold
- Type `z E` for eliminate to remove all folds
- Type `z M` for more to close all folds
- Type `z R` to open all folds

As you can imagine `ex` allows more possibilities to define a region to fold. The command looks like this:

```
:<range> fold
```

A range can be

```
lines :15,20 fold
```

```
search :?start?,/end/ fold
```

```
marks :'b,'c fold
```

```
combinations :'b,/pat/ fold or :23,'c fold or ...
```

To do all the folding by hand is possible and has to be saved for future use. If you don't save the manual folds, they are lost if you close and reopen the file again. Save your folds with:

```
:mkview
```

Load your previously saved folds with:

```
:load
```

Vim provides also an automatic folding depending on indentation, which is useful for Python or YAML, but also on syntax and on differences, if you want to compare to files. Check the help for more.

Check for the folding method with following command:

```
:set foldmethod
```

Most interesting methods are `manual`, `indent`, `syntax`, `diff`.

Fold summary

keys	description
:set foldmethod=manual	manual folding
:mkview	store folds
:load	load folds
5zf	fold 5 lines
zo or 	open fold
za	toggle fold
zc	close fold
zd	delete fold
zE	eliminate all folds
zM	close all folds
zR	open all folds

For more information type **:h fold**

8.4 Key combinations to think about

You can combine a movement with a command. Let's say you want to delete the next two words in your line.

   or    

will do it. Literally, it deletes two times a word. As an alternative you can try the following:

It will do the same by deleting two words at once or you can type:

It will do the same by deleting one word and repeating the last action. But there is a difference, if you want to undo these three different actions above. With the last action you have to type u twice.

You can further do this to delete two times two words:

Ok, I don't see a use case to do so, but it demonstrates how Vim is working.

will insert 100 times 0. Note that any numbered insertion will take effect after leaving the `insert` mode with . In fact there is a use case described in section 8.11 on page 59. Look at step 5.

8.5 Magic *g*

If you have played around with regular expressions and replacements, I guess you made the experience that's complicated and sometimes it does not work as expected. Fortunately you can hit `u` to undo the last missed replacement.

With the command `g&` you have a way to test your regular replacement string locally on one line and if it fits finally you can do it with this `g`-command for the whole buffer again. The command `g&` repeats the last `:s///` replacement for the complete buffer.

Sometimes you have very long lines. You see this, if you have line numbers. There will be a gap in the numbering even if you see lines. With the setting `nowrap` you have to scroll horizontally.

The motion commands in Vim preceded by `g` let you go into the wrapped long lines. Try yourself the difference between `$` and `g$` on long lines. The same applies for `j` or `k`.

Have you used `I` to insert at the begin of a line? The cursor will be placed on the starting non white space character of the line. If you want to insert a commenting character like `%` in `LaTeX` or `#` in shell scripts or `"` in your `.vimrc` you may want to be in column 1. Type `gI` it will do what you want.

We talked earlier about long lines and formatting. Using `J` to concatenate lines is sometimes useful. I often do afterwards `gq` to insert line breaks again. For normal text it makes sense to insert a space for the former line breaks. But if you work on other pieces of text, you may don't want these inserted spaces per `J`. Type `gJ` and the lines are concatenated without spaces.

May be a rather rarely useful usage of the `g` command is to append something to a matched line. It could be useful in tables. Type `:g/pat1/norm A pat2` and `pat2` will be appended to all lines which match `pat1`. `norm A` switches to normal mode and `A` is the command for append at the end of the line.

g summary

keys	magic
g&	repeat last :s/// for whole buffer
g\$	go to end of wrapped line
gj	go one line down in wrapped line
gk	go one line up in wrapped line
gI	go first column in line
gJ	concatenate without spaces
:g/pat1/norm A pat2	append pat2 in all lines with pat1

For more information type **:h g**.

8.6 Pasting search results

With the command `:g/<pattern>/p` the search result is shown below the status line. The trick is to yank the output into a register. To be more specific to append to a named register. You have to use the capital letter of one of the named registers a till z.

So, with some small modification like this:

```
:g/<pattern>/y A
```

each yanked line of the output is appended in the named register a. You now can paste its contents with:

```
"ap
```

8.7 Reading external stuff

From time to time it is useful to insert some external stuff into your documentation. Imagine you want to have a list of files of your current directory documented in you file.

Vim provides this interface to read the output of external programs.

Type

What does it do? With `:` you enter command mode, `r` will read into the file, `!` enters the command line of your OS and `ls` is the command to list the files in your directory.

The output is put at the recent cursor position. `ls` can be replaced by any other meaningful OS command, e.g. with `pwd` to read the working directory or with `cat <file>` to read a file. You will find much more.

8.8 Repeating commands

On one hand side I search often, but on the other hand I search for other things the second time. If you often search for the same, you can repeat your search like this:

Initial search: `/patTern` searches for `patTern`

Repeated search: `//` searches again for `patTern`

More powerful is the `.` change or yank repetition. Let's make an example.

```

21 | column 1 | column 2 | column 3 |
   | :----- | :-----: | ----- |
   | bla 1    | bla 2    | bla 3    |
   | blb 1    | blb 2    | blb 3    |
   | blc 1    | blc 2    | blc 3    |
   | bld 1    | bld 2    | bld 3    |

```

`table_example.md`

21, 24

I want to change all `blx` with `bulx`.

1. I search for `bl` and type `/ b l ↵`
2. I append `u` after `b` and type `a u`
3. I go to the next by typing `n`
4. and I repeat the insertion with `.`
5. I continue typing `n . n . n . ...`

And now an example for a yank repetition.

8.9 Special deletions

From time to time I copy a file as a new chapter and want to keep some lines at the top, but remove the rest till the end. This is very handy with `vim`. You don't have to select the text, you simply type:

`d ↑ + g`

It deletes all lines from where the cursor is till the end of the file.

d g g

This deletes all line from where the cursor is till the top of the file. As a programmer I have to deal with any kind of brackets very often. Imagine following piece of code:

```
\tikz [baseline=-4.5pt] \node [key] {d};
```

I want to remove the first square brackets. There are many ways to do so. A fascinating one is to move the cursor somewhere inside the brackets and to type:

d a [or **d a]**

This also works with other brackets () { } < > and " or '. But that's not all folks. You can use other Vim objects like words, sentences or paragraphs.

d a p
d a t

would delete the paragraph p you are in with your cursor respectively the XML/HTML-Tag t.

If you want to replace the contents within the brackets, use **i** instead of **a**. i stands for *inside*, while a may be interpreted as *all*, including surroundings.

8.10 Special yanking

The yanking commands include always the character or line of the cursor position. Below are some example shown. Yanking allows any motion in Vim. I guess most impressive is the fact to yank with the smallest number of key strokes a complete paragraph or the whole document.

key strokes	action
y w	yank the characters till end of word
y y	yank the line
3 y y	yank three lines
y }	yank the paragraph
g g y G	yank the complete document
y / <pattern> <↵>	yank till pattern
y 2 / <pattern> <↵>	yank till the second occurrence of pattern

8.11 Tricky tabulators

I sometimes face the problem that I copy text from various sources and lose the tabulators. This happens for example, if you copy a table from the browser. On the browser the table can be read well, but in the text file you lose this structure.

Let's look to an example.

```
42 SELECT
1 ID BIGINT NOT NULL GENERATED BY DEFAULT AS IDENTITY,
2 CREATION_DATE TIMESTAMP NOT NULL,
3 PROJECT_NAME VARCHAR(256),
4 SENDER VARCHAR(256),
5 RECEIVER VARCHAR(256),
```

table_example.md

42,1

How can I get the definitions of the database columns aligned properly?

Step 1: Check for the longest column name. It's CREATION_DATE. The column to be used for alignment should be 16, then go to line 43.

Step 2: Start recording a macro, which shall do the job. I have chosen a as name for the macro.

qa

Step 3: Go to the start of the line.

0

Step 4: Find the delimiter between column name and definition. It's a space.

f_

Step 5: Insert a lot of spaces as a preliminary step.

```
40i_
```

Step 6: Go to the desired column 16 and remove the remaining spaces.

```
16|dw
```

Step 7: Go to next line and quit recording.

```
jq
```

Step 8: Execute the macro a for the remaining 5 number of lines.

```
5@a
```

The macro all in one: `qa0f_40i_16|dwjq`

```
42 SELECT
1 ID BIGINT NOT NULL GENERATED BY DEFAULT AS IDENTITY,
2 CREATION_DATE TIMESTAMP NOT NULL,
3 PROJECT_NAME VARCHAR(256),
4 SENDER VARCHAR(256),
5 RECEIVER VARCHAR(256),
```

`table_example.md`

`42,1`

Refer to <https://stackoverflow.com/questions/6154306/how-to-insert-spaces-up-to-column-x-to-line-up-things-in-columns/8363443#8363443>
or also to https://vim.fandom.com/wiki/Simple_text_alignmen.

8.12 Using the terminal

The developers of Vim have anticipated that you want to interact with the outside world while you are in Vim. There is a convenient way to do so.

Type

```
:term
```

and you get a split window with your terminal. Now type any shell command and see the result.

On my Mac I get the Z Shell `zsh` and on my Linux box the Bourne-Again Shell `bash`. This is useful as you can do many things. You can compile, search, anything you usually do in your shell. But the really amazing feature of Vim is to switch the terminal into a buffer. Type

```
ctrl-w N
```

and the terminal converts into a buffer with all features. I guess the most interesting feature is to yank some output of your shell into your document in the other window split.

To get out of the buffer mode hit `i` or `a` and you can go on executing shell commands. Leave the terminal with `exit`, as you usually close your shell.

8.13 Visual block mode

I remember the times when I was working on the IBM mainframe. I had to create high quality technical documentation with help from DCF¹. Here I learned the first time to manipulate columns in line oriented text. It was possible to select portions of a table and move them. Previously, during my studies I was not aware of such feature. I used WordPerfect to create my diploma thesis.

Now, Vim has this feature and it is very powerful. In markdown I often work with tables and they look like this:

```
21 | column 1 | column 2 | column 3 |
  1 | :----- | :-----: | ----- |
  2 | bla 1    | bla 2    | bla 3    |
  3 | blb 1    | blb 2    | blb 3    |
  4 | blc 1    | blc 2    | blc 3    |
  5 | bld 1    | bld 2    | bld 3    |
```

table_example.md

21, 24

What can I do, if I want to copy column 3 and paste it again before column 3?

Step 1: place the cursor on the blank before column 3 as shown.

Step 2: type `ctrl+v` for visual block mode.

Step 3: type `5j` to go to the last line of the table. You see how relative numbering helps.

Step 4: type `$` to select column 3. It moves the right end selector to the end of the line.

Step 5: type `y` to yank the selection.

¹Document Composition Facility

Step 6: type  +  to paste the selection before the cursor position.

There are many other ways to do so, you can e.g. enter visual block mode and move around with the move keys , , ,  until your selection is fine. Moreover, the borders of visual block can be enhanced or reduced in all directions at any time, if you press  during your movements. With  you switch from the top left edge to the bottom right edge and vice versa.

You can also add new text to a column in an elegant way. With the actions above you got following table with four columns:

5		column 1		column 2		column 3		column 3	
4		:-----		:-----:		-----		-----	
3		bla 1		bla 2		bla 3		bula 3	
2		blb 1		blb 2		blb 3		bulb 3	
1		blc 1		blc 2		blc 3		bulc 3	
26		bld 1		bld 2		bld 3		buld 3	

table_example.md
26, 47

Now we want to replace the number 3 by 4.

Step 1: place the cursor on the second 3 in line 21.

Step 2: type   and replace the 3 by 4.

Step 3: place the cursor on the second 3 two lines below.

Step 4: type  +  for visual block mode.

Step 5: type   to select all 3.

Step 6: type   to replace all 3 by 4.  +  does the same.

Now we want to insert some text.

Step 1: place the cursor on the last l in line 23.

Step 2: type  +  for visual block mode.

Step 3: type   to select all l.

Step 4: type  +  to get into insert mode and type .

Step 5: type  to leave visual mode.

With the last  the magic happens and on all lines the u is inserted and a bulb appeared.

For completeness I'll mention the other two variants of the `visual` mode. There is a rather intuitive way to select text visually by typing `v` followed by some motion.

`vw`

will draw a Volkswagen. :-) no, it selects a word, starting from the cursor position. Any motions are possible. `vip` selects a paragraph.

If you want to visually select complete lines, you can type `↑+v`. This works at any position on the line and you can switch between these selection modes by typing `v` or `V` or `ctrl+v`.

Visual modes summary

keys	description
<code>vw</code>	visually select word
<code>v3w</code>	visually select three words
<code>vip</code>	visually select inner paragraph
<code>V</code>	visually select a line
<code>ctrl+v</code>	visual block mode
<code>o</code>	switch the ends of the selection
	leave visual mode

8.14 Word completion

Vim does word or line completion like many integrated development environments. You can define what sources shall be used in the `.vimrc` file. Refer to section 9 on page 67.

The completion works in `insert` mode. If you are start typing and then type + or + you'll get a list of suggestions for completion. Chose one from the drop down list and press `↵`. You stay in `insert` mode and you can continue writing.

If you want to repeat a complete line, you start typing like the line you want to complete and then type + +. You'll get a list of lines with the same beginning or all lines, if haven't typed anything. Chose one from the drop down list and press `↵`.

Completion summary

keys	motion
ctrl+n or ctrl+p	Get a list of words depending on the flags in your .vimrc file.
ctrl+x ctrl+n	Words of the current buffer. This may shorten the list, if you have many flags activated in your .vimrc file.
ctrl+x ctrl+l	Complete lines, which start with the pattern you have already inserted.
ctrl+y	Accept completion with yes
ctrl+e	Exit the completion
ctrl+h and ctrl+l	Go backward and forward with your completion

For more information type **:h i_ctrl-n**.

9 My configuration

```
1  " ----- "
2  " My Vim Run Commands file "
3  " Harald Lichtenstein      "
4  " 2025-10-24              "
5  " ----- "
6  autocmd FileType tex setlocal spell
7  " Some *.tex-files were detected as plaintext,
8  " which has other highlighting colors.
9  autocmd BufRead,BufNewFile *.tex set filetype=tex
10 autocmd InsertEnter * set nocursorline
11 autocmd InsertLeave * set cursorline

13 colorscheme torte

15 "Cursor settings:
16 " 1 -> blinking block
17 " 2 -> solid block
18 " 3 -> blinking underscore
19 " 4 -> solid underscore
20 " 5 -> blinking vertical bar
21 " 6 -> solid vertical bar
22 let &t_SI.="\"e[5 q" "SI = INSERT mode
23 let &t_SR.="\"e[2 q" "SR = REPLACE mode
24 let &t_EI.="\"e[1 q" "EI = NORMAL mode (ELSE)

26 set autoindent
27 set clipboard=unnamed
28 set cursorline
29 set complete=.,w,b,u,t,i,kspell
30 " Following line deletes the last search register "/"
31 let @/ = ""
32 set hlsearch
33 set incsearch
34 set laststatus=2

36 " unicodes: → := <ctrl-v>u2192
37 "           ␣ := <ctrl-v>u2423
38 set listchars=tab:→\ ,eol:$,space:␣
39 set number
40 set relativenumber
41 set scrolloff=5
```

```
42 set shiftwidth=3
43 set nospell
44 set spelllang=en
45 syntax spell toplevel
46 set tabstop=3
47 set textwidth=80
48 set ttyfast
49 set wildmenu
50 syntax on
51 highlight Visual ctermfg=Black ctermbg=Cyan
52 highlight CursorLine cterm=underline ctermbg=Black ctermfg=none
53 highlight Search cterm=NONE ctermfg=black ctermbg=yellow
54 set foldcolumn=3
55 iabbrev tt \texttt{
```

The first thing, which seems to be obvious: double quotes " introduce a comment in `.vimrc`. The first non commenting lines start with `autocmd`. For these commands Vim listens to the related events and executes the command, if this event occurs. Note that the order matters, some entries depend on others. I experienced this effect especially with the highlighting.

Line 6: Activate spelling `spell` for the current buffer only `setlocal`, if the file type `FileType` is `TeX tex`.

Line 9: Set the file type to `TeX filetype=tex`, if a file with ending `tex` is read or opened into a buffer `BufRead, BufNewFile *.tex`.

Line 10: Deactivate the line under the row `set nocursorline` where the cursor is, when `insert` mode is entered `InsertEnter`. Do this independent of the file type `*`.

Line 11: Highlight the row where the cursor is `cursorLine`, when leaving the `insert` mode `InsertLeave`. Also valid for all file types `*`.

Line 13: Chose highlighting color scheme named `torte`

Line 22: The lines 22 to 24 manipulate the cursor look, depending on the mode. The intention is to better see whether you are in `insert` mode or not. If it works, depends on the terminal you use, because it is not a feature of Vim but of the terminal. You will get different results, depending on your OS. Try yourself.

Line 23: Use a block cursor in `replace` mode.

Line 24: Use a block cursor in `normal` mode.

-
- Line 26:* Let Vim indent a new line depending on the previous line.
- Line 27:* This allows Vim to use the clipboard of the OS. Try the difference between unnamed and unnamedplus. In Linux I use to insert the selection with middle mouse button. + and + also works.
- Line 28:* Highlights the row where the cursor is.
- Line 29:* Activates the autocomplete function in Vim. In insert mode press + or + and Vim tries to complete the word. The flags control where the suggestions come from. The flag `.` stands for the current buffer, `w` for the windows, `b` for other loaded buffers, `u` for unloaded buffers, `t` for tags, `i` for included files and `kspell` for the dictionary.
- Line 31:* As written the search register will be deleted. This has the effect that there is no highlighted text from the last search.
- Line 32:* All occurrences, which match the search pattern are highlighted.
- Line 33:* As soon as you type  followed by a character Vim will start searching and with the line above highlight the result.
- Line 34:* Sets the status line as always visible.
- Line 38:* If you want to show invisible characters like tabulators or spaces, you can type `:set list`. This command modifies the look of the standard ones. The tab will be an `→` and the space a `␣`.
- Line 39:* Show line numbers.
- Line 40:* Show the line numbers as relative line numbers. This shows the real line number of the line where the cursor is and counts from this position up and down the number of previous respectively subsequent lines.
- Line 41:* This setting leaves the specified number of lines on top and at the bottom of the file, if you scroll up or down.
- Line 42:* This setting defines how many spaces are used for autoindentation or manual indentation. The manual indentation uses following key strokes:   or   or   or  . All can have a number as prefix.
- Line 43:* This line deactivates the highlighting of misspelled text in Vim.

- Line 44:* Use an English dictionary, if the spell check is active. There are many languages available. For German use `de` instead of `en`.
- Line 45:* This command is useful, if you write pure text or $\text{\LaTeX}$, because all parts are included (`toplevel`) during spell checking. Else only comments are looked at.
- Line 46:* An inserted real tabstop (`\t`) shall occupy a 3 spaces distance.
- Line 47:* With this setting `Vim` inserts a linebreak after 80 characters more or less. You avoid very long lines, which would exceed the window width. This is typically useful for $\text{\TeX}$ and $\text{\LaTeX}$, which does its own linebreaking during compilation.
- Line 48:* If you don't use a terminal over a slow modem connection and sit in front of a modern PC, you may set this to tell `Vim` to send more characters at once, which is faster. Scrolling is smoother with it for example.
- Line 49:* This stands for `wildcard` menu and allows to expand your typing in the command line. Type  to go to the command line and start typing, then press  and you will get completion suggestions, continue pressing  to go to the next proposal.
- Line 50:* Enables syntax highlighting.
- Line 51:* Defines my style of selecting text in `visual` mode. The text color is black and the text background is cyan. `ctermfg` stands for `color terminal foreground`. As you might expect `bg` stands for `background`.
- Line 52:* This line in the configuration file creates a thin underline of the row where the cursor is. The text background is black and there shall be no foreground color. The black background fits to the `torte` color scheme, which is also black.
- Line 53:* This line sets the highlighting of search results. The background of the color terminal yellow with letters in black, which is the foreground color.
- Line 54:* This line sets the number of fold levels visible in the left most column.
- Line 54:* This line defines the abbreviation `tt` for insert mode as `\texttt{}`.

10 Bibliography

- [1] ARNOLD ROBBINS and ELBERT HANNAH, Learning the vi & Vim Editors, 8th Edition, O'Reilly Media, Sebastopol, California, 2021
- [2] DREW NEIL, Practical Vim, Edit Text at the Speed of Thought, The Pragmatic Programmers, The Pragmatic Bookshelf, Dallas, Texas, Raleigh, North Carolina, 2012
- [3] JOHN BAMBENEK and AGNIESZKA KLUS, grep Pocket Reference, O'Reilly Media, Sebastopol, California, 2009
- [4] JEFFREY E. F. FRIEDL, Mastering Regular Expressions, Third Edition, O'Reilly Media, Sebastopol, California, 2006
- [5] ERIC STEVEN RAYMOND, The Art of Unix Programming, <https://cdn.nakamotoinstitute.org/docs/taoup.pdf>, 2003
- [6] DALE DOUGHERTY and ARNOLD ROBBINS, sed & awk, Second Edition, O'Reilly Media, Sebastopol, California, 1997
- [7] STEVE OUALLINE vimbook-OPL.pdf, <https://mirror.math.princeton.edu/pub/vim/doc/book/>